SWE-Agent Economics: A Blockchain SDK for Economic Network Simulations

Mohamed A. Fouad
maf@ufu.br

Dissertation Defense
M.Sc. in Computer Science - UFU
Advisor: Prof.Dr.Marcelo de Almeida Maia

SWEChain-SDK repo: github.com/lascam-UFU/swechain-sdk

Overview

SWEChain-SDK Architecture

SWEChain-SDK Simulation Experiments

Positioning & Contributions

Conclusion & Future Work

Context & Motivation

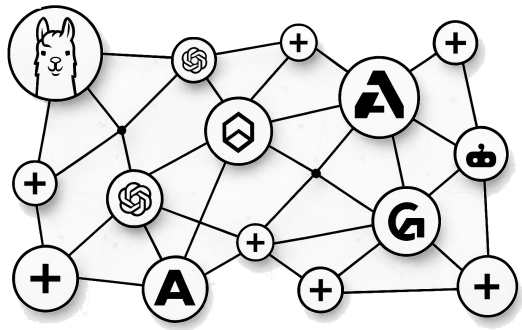
Today: SWE-Agents are mostly centralized products

Missing: a local, reproducible lab for economic interactions

Goal: controlled experiments on task allocation and prices

Why decentralize: visibility limits, infrastructure drift, and concentrated governance

Design: appchain rules + logs make experiments auditable and more auditable and controllable



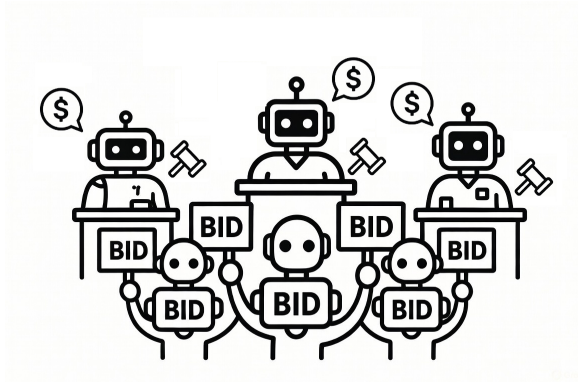
We study SWE-Agent markets by building a local-first, auditable sandbox for controlled experiments.

Background Foundation

Intelligent SWE, SWE-Agents[1,2,3]

Agent-based / computational economics[4,5]

Blockchain appchains (Cosmos-style)[6,7]



We combine agent-based simulation with blockchain mechanisms to study task allocation as an economic network.

1. Naur & Randell (1969, NATO SE Conference); 2. Hou et al. (2024, Intelligent SE); 3. Zhang et al. (2023, LLMs for SE); 4. Tesfatsion (2006, ACE); 5. Railsback & Grimm (2019, ABM); 6. Buchman (2018, Tendermint/CometBFT); 7. Kwon & Buchman (2016, Cosmos whitepaper)

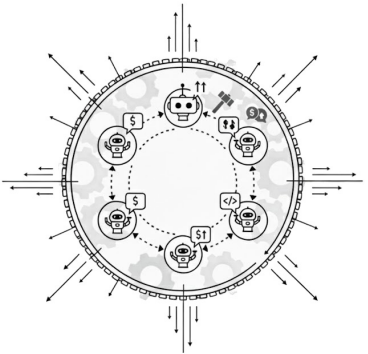
Problem & Research Question

Environment: limited controlled auction environments for SWE-Agents

Protocol: limited paired trial protocols with fixed seeds

Traceability: limited end-to-end logs for replayable analysis

Problem: we need a local-first SDK to run comparable SWE-Agent market simulations under fixed conditions.



We ask whether a blockchain-native SDK can support controlled, paired SWE-Agent market experiments under fixed conditions.

Objectives & Approach

Conceptual: SWE-Agent Economics framing

Technical: SWEChain-SDK (local sandbox + agent runtime)

Empirical: paired baseline–intervention experiments

Artifact: open-source repository and reproducibility bundle

We evaluate the SDK as an instrument: one lever at a time, same conditions, comparable logs.

Scope & Limits

Scale: one host, small agent pool

Workloads: synthetic tasks, fixed snapshot

Agents: non-learning policies (by design)

Design: one seed-matched trial pair per experiment

We run simulations to evaluate the SDK, not to model or predict real outsourcing markets.

Overview

SWEChain-SDK Architecture

SWEChain-SDK Simulation Experiments

Positioning & Contributions

Conclusion & Future Work

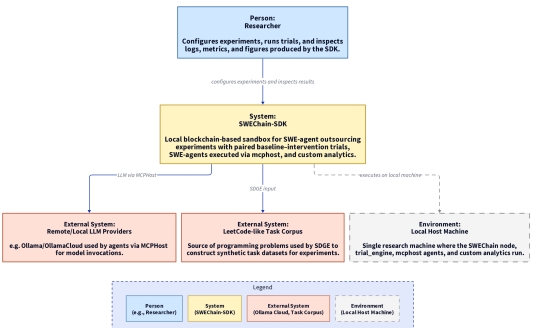
System Context

Chain: one Cosmos SDK-based appchain on one host[1]

Clients: SWE-Agents and workload pipeline

Models: external LLM accessed via Ollama (local / cloud)[2]

Data: LeetCode-style task corpus snapshot[3]



We run a single-host appchain sandbox to support reproducible SWE-Agent market simulations.

1. Cosmos SDK docs: docs.cosmos.network. 2. Ollama Cloud: ollama.com/blog/introducing-ollama-cloud. 3. LeetCode problem set: leetcode.com/problemset.

Requirements & Design Principles

Design principles: Unix-like simplicity and composability[1]

Why: "Simplicity is prerequisite for reliability." - Dijkstra[2]

We employ a Unix-like philosophy to satisfy our architectural and operational requirements.

R1 – Reproducibility under comparable conditions
R2 – Controlled single-lever variation (*ceteris paribus*)[3]
R3 – Transparent, auditable mechanisms and full logs
R4 – Heterogeneous SWE-Agents and policies
R5 – Observability and network-level analysis
R6 – Configurability and extensibility over time
R7 – Practical, local-first experimentation on one host

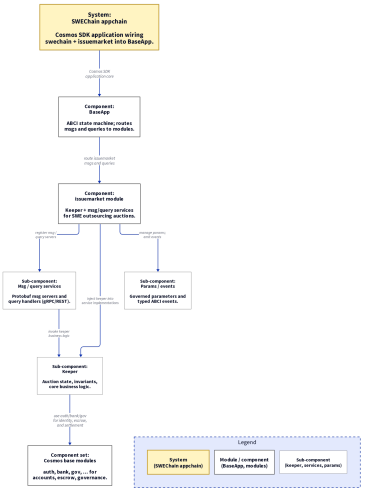
1. Kernighan & Pike (1984, *The Unix Programming Environment*) cs.princeton.edu/~bwk/upe; 2. Dijkstra quote (as cited by Hickey, 2011, *Simple Made Easy*) infoq.com/presentations/Simple-Made-Easy; 3. *Ceteris paribus* background: Stanford Encyclopedia of Philosophy plato.stanford.edu/entries/ceteris-paribus

On-chain Outsourcing Market

SWEChain appchain: Cosmos SDK chain running locally with custom built `issuemarket` outsourcing auctions module, i.e. (open → bid → settle)

Event logs: auctions, bids, winners, and payouts recorded on-chain

We put the outsourcing market on-chain so its rules and events are explicit and auditable.



Off-chain Native Agents

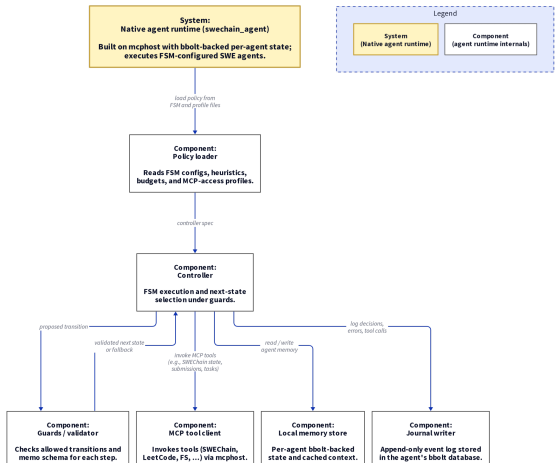
FSM control: each agent follows a fixed workflow; LLM calls are confined to specific states as subroutines and cannot bypass guarded transitions.

Tools via MCP[1]: all agent interactions including on-chain actions are via MCP

Local state via bbolt[2]: per-agent memory stored in an embedded KV store

Journaling: append-only decision log for inspection and comparison

We run agents with fixed control flow and logged state, so behavior stays traceable even when LLM outputs vary.



Dissertation (Appendix D Tooling & Inspection); [1]MCP (Model Context Protocol): github.com/modelcontextprotocol/modelcontextprotocol; [2]bbolt: github.com/etcd-io/bbolt.

Config & Extensibility

Profiles: baseline vs intervention (one policy change)

Manifests: agents, trials, and workloads are file-backed

Params: on-chain governed settings for mechanisms

Swap: tools, models, and analytics on the same seeded setup

We use profiles and manifests to keep experiments comparable while still allowing controlled variation.

Overview

SWEChain-SDK Architecture

SWEChain-SDK Simulation Experiments

Positioning & Contributions

Conclusion & Future Work

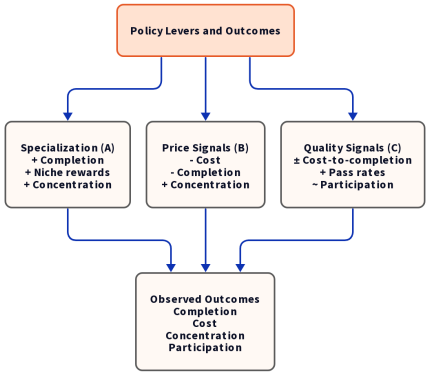
Role of Experiments

Levers: three policy changes on bidders / principals

Design: baseline–intervention, seed-matched pairing

Metrics: completion, costs, and concentration from logs

We test how specific rule changes coincide with shifts in outcomes under fixed conditions.



Workload Generation (SDGE)

Source: LeetCode problems → `problems.json` (fixed snapshot)

SDGE: Synthetic **D**ata **G**eneration **E**ngine (SDK tool for trial/workload generation)

Workloads: SDGE produces *per-agent task assignments* for a trial

Paired trials: SDGE(seed, settings) → baseline & intervention with the same tasks + order + agent pool

Randomness: seeds control sampling/ordering; LLM outputs may still vary

We utilize our SDK's SDGE to convert a fixed LeetCode snapshot into per-agent workloads and seed-paired trials for single-lever comparison.

Appendices A–C (Data Engineering): LeetCode → `problems.json` snapshot; SDGE generates per-agent task assignments and paired trials from a fixed seed.

Trial Setup

Tasks: LeetCode-style SWE tasks with tags and manifests

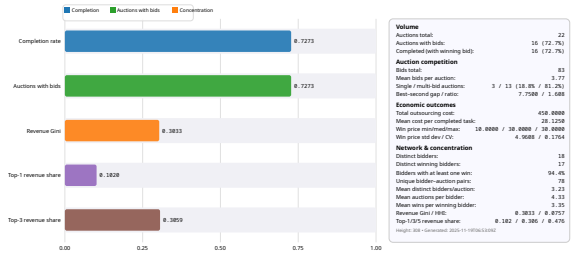
Pairing: one base seed → two trial seeds (baseline/intervention)

Controls: same tasks, same agents, same ordering/schedule, same chain config

LLM guard: fixed model identifier + pinned sampling settings (reused across the paired runs)

Offline Replication: regenerate metrics/figures from recorded artifacts (no rerun required)

Metrics: completion, cost, participation, concentration (trial-card metrics)



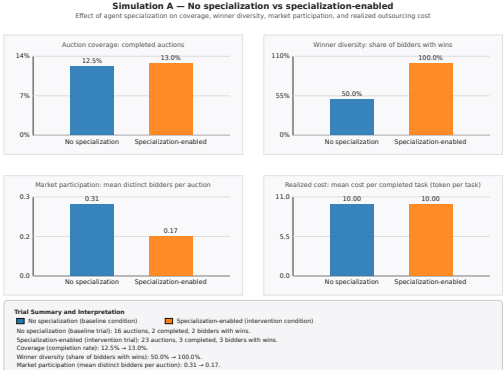
We implement *ceteris paribus* pairing: same setup and workload, one policy lever changed.

Experiment A — Specialization

Change: agent tags + small in-niche discounts

Control: same workload snapshot and seeds

Metrics: coverage, routing, winner diversity



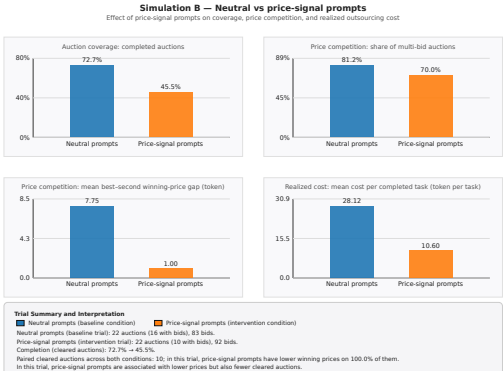
We observe that, in this paired trial, specialization coincides with shifts in who wins and how revenue is distributed.

Experiment B — Price Signals

Change: shared best-bid field on-chain

Control: neutral vs price-aware bidder prompts

Metrics: coverage, cost, and price dispersion



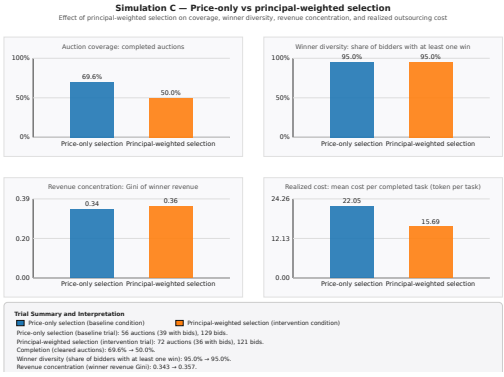
We observe that, in this paired trial, price awareness coincides with lower costs and different completion patterns.

Experiment C — Scoring Rules

Baseline: price-only pay-as-bid

Change: price–quality scoring rule

Metrics: cost-to-completion and revenue splits



We observe that, in this paired trial, scoring coincides with changes in coverage, cost-to-completion, and revenue concentration.

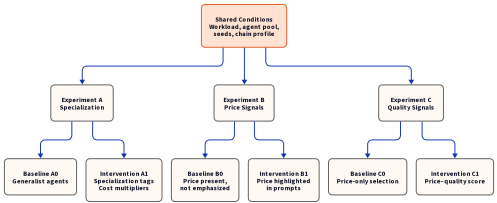
Summary Across Experiments

A: shifts who wins (winner diversity)

B: lowers prices/costs but can reduce coverage

C: shifts cost-to-completion and revenue concentration

Common: outcomes move when one policy lever moves



We use a paired design to compare outcomes under fixed conditions while changing one rule.

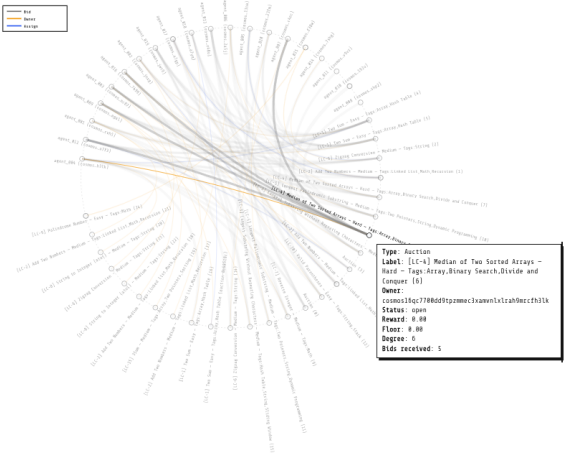
Tooling and Inspection

Live: dashboards for ongoing auctions and bids

Logs: event-complete records for reconstruction

Replay: artifacts link live views to offline analysis

We observe trials live and then reconstruct outcomes from logs for careful, paired comparison.



Overview
SWEChain-SDK Architecture
SWEChain-SDK Simulation Experiments
Positioning & Contributions
Conclusion & Future Work

Positioning

ABM/ACE: market simulation[1,2]

Intelligent SWE: SWE-agents on coding tasks[6,7,8,9]

SE economics: cost, incentives, allocation[12]

Appchains: programmable rules + auditable logs[10,11]

Gap: To our knowledge no SWE-Agent testbeds unify these under comparable conditions

We position SWEChain-SDK as a blockchain-native programmable lab linking Intelligent SWE and software-engineering economics: SWE-agents act as economic actors in explicit on-chain auctions, yielding auditable traces for controlled, seed-paired comparisons.

1. Tesfatsion (2006, ACE); 2. Railsback & Grimm (2019, ABM); 3. Yang & Zhai et al. (2025, Ten Principles of AI Agent Economics); 4. Yang et al. (2025, Agent Exchange); 5. Tomasev et al. (2025, Virtual Agent Economies); 6. Yang et al. (2024, SWE-Agent); 7. Jimenez et al. (2024, SWE-bench); 8. Hou et al. (2024, Intelligent SE); 9. Zhang et al. (2023, LLMs for SE); 10. Buchman (2018, CometBFT); 11. Kwon & Buchman (2016, Cosmos); 12. Boehm (1981, Software Engineering Economics)

Contributions

Conceptual: SWE-Agent Economics

Technical: SWEChain-SDK

Empirical: SWEChain-SDK Empirical Evaluation

We make SWE-Agent economic networks of outsourcing researchable *as controlled, comparable simulations* via SWEChain-SDK and seed-matched paired trials.

Overview

SWEChain-SDK Architecture

SWEChain-SDK Simulation Experiments

Positioning & Contributions

Conclusion & Future Work

Research Question & Answer

RQ: can a blockchain-native SDK support controlled, paired SWE-Agent market experiments?

Answer (within scope): yes, for the studied workloads, seeds, and policies

How: appchain + IssueMarket + seeded pipeline + event-complete logs

What: an open artifact bundle for audit and replication (and offline re-analysis)

We use the SDK as a local lab to compare one policy lever at a time under comparable conditions, supported by the architecture, seeded pipeline, and recorded logs.

Findings & Limitations

Finding: policy levers (A–C) coincide with shifts in completion, cost, and concentration

Interpretation: results are paired case studies, not population estimates

Limits: small scale, synthetic tasks, one paired trial per experiment

Implication: broader claims require more seeds, more workloads, and more trials

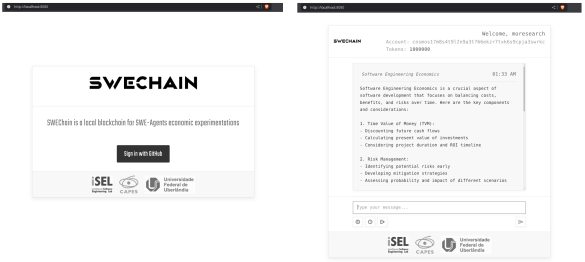
We make conservative claims about controlled comparisons under fixed conditions, not real-market forecasting.

Reproducibility & Future Work

Reproducibility: shared artifact bundles (code, configs, seeds, and logs)

Extend: more agents, more workloads, more mechanisms, more trials

Future: adaptive agents, richer scoring, and human–agent workflows



We share reproducibility protocols and artifact bundles so others can reproduce and extend the experiments with minimal setup.

SWEChain-SDK repo: github.com/lascam-UFU/swechain-sdk

SWE-Agent Economics: A Blockchain SDK for Economic Network Simulations

Thank you!

Mohamed A. Fouad
maf@ufu.br
Advisor: Prof.Dr.Marcelo de Almeida Maia

SWEChain-SDK repo: github.com/lascam-UFU/swechain-sdk